

Walk-in & Queue Management SaaS Platform

Developer: Ahmed Hamid

Email: ahmedhamid2000422@gmail.com

Date: 25 May 2026

Live Deployment: <https://azqueue.io>

GitHub: github.com/ahmedhamid2000422-eng/azqueue-saas-app

Full-Stack SaaS	16 DB Migrations	7 Edge Functions	Live at azqueue.io
React + Supabase	Row Level Security	Deno/TypeScript	Vercel Deployment

1. Executive Summary

AzQueue is a production-deployed, full-stack Software-as-a-Service (SaaS) platform built to replace paper ticketing and disconnected booking calendars used by walk-in service businesses. The system manages the complete customer journey from check-in to completion, with real-time queue management, AI-powered customer intelligence, multilingual support, WhatsApp/SMS notifications, and automated satisfaction scoring.

The platform is live at azqueue.io, deployed on Vercel (frontend) and Supabase Cloud (backend), with a PostgreSQL database structured across 16 sequential migrations and 7 Deno Edge Functions handling server-side logic including Twilio WhatsApp messaging, Stripe billing, and OpenAI AI persona generation.

Aspect	Detail
Project Type	SaaS product — multi-tenant, subscription-based
Target Market	Multilingual walk-in service businesses (clinics, law firms, government offices, salons)
Development Period	Jan 2025 – May 2025 (active development, live in production)
Lines of Code	~12,000 lines across JSX, JS, TypeScript, SQL
Database Tables	16 migrations, full RLS security on all tables
External APIs	OpenAI GPT-4o-mini, Twilio WhatsApp, Meta Cloud API, Stripe, Freshdesk
Deployment	Vercel (auto-deploy from GitHub main branch) + Supabase Cloud
Live URL	https://azqueue.io

2. System Architecture

AzQueue follows a client-heavy architecture where the React frontend communicates directly with Supabase via the Supabase JavaScript client, using Realtime subscriptions for live queue updates. Heavy server-side operations (messaging, billing, AI) are offloaded to Supabase Edge Functions which run on Deno at the edge.

Layer	Technology	Role
Frontend	React 18 + Vite + Tailwind CSS	SPA with React Router v6, real-time queue UI, kiosk check-in pages
Auth	Supabase Auth (JWT)	Email/password sign-in, session management, RLS enforcement
Database	PostgreSQL (Supabase)	16-table schema with Row Level Security on every table
Realtime	Supabase Realtime (WebSocket)	Live queue updates pushed to all connected staff dashboards
Edge Functions	Deno (TypeScript)	7 functions: messaging, billing, AI, admin stats
AI	OpenAI GPT-4o-mini	Customer persona generation, AI reply drafting
Messaging	Twilio WhatsApp API	Queue notifications: confirm, call, thanks, prayer pause
Alt Messaging	Meta WhatsApp Cloud API	Direct channel for businesses with own WA Business number
Payments	Stripe	Subscription checkout, customer portal, webhook processing
CRM	Freshdesk REST API	Pulls open support tickets into customer profiles
Hosting	Vercel	CDN-deployed frontend, auto-deploy on GitHub push
Domain	azqueue.io	Purchased and DNS-configured via Vercel

Customer checks in via QR code → CustomerCheckIn.jsx calls findOrCreateCustomer() → ticket inserted to DB → Supabase Realtime pushes update to Queue.jsx on staff dashboard → staff sees customer persona card → on completion, send-notification Edge Function sends Twilio WhatsApp → satisfaction survey shown to staff → score saved to satisfaction_scores table.

3. Technology Stack

Technology	Version/Detail	Purpose
React	18.x	Frontend UI framework
Vite	5.x	Build tool and dev server
React Router	v6	Client-side routing and navigation
Tailwind CSS	3.x	Utility-first CSS, custom design tokens
Supabase JS	2.x	Database, Auth, Realtime client
PostgreSQL	15 (via Supabase)	Primary database with RLS
Deno	1.x	Edge Function runtime (TypeScript)
OpenAI API	GPT-4o-mini	AI persona generation and reply drafts
Twilio	WhatsApp Business API	Outbound customer notifications
Stripe	Latest API	Subscription billing and customer portal
Freshdesk	REST API v2	CRM ticket enrichment on customer profiles
Fraunces	Variable font	Display typography (headings)
Inter	Variable font	Body and UI typography
JetBrains Mono	Latest	Monospace / ticket numbers
Vercel	Hobby/Pro	Frontend deployment with CDN
GitHub Actions	Via Vercel integration	CI/CD pipeline

4. Database Schema — 16 Migrations

The database is structured as sequential SQL migrations, each adding tables, indexes, and Row Level Security policies. Every table enforces branch-level data isolation — staff can only read/write their own branch's data.

Migration	Tables Created	Purpose
0001_init.sql	branches, staff, services, tickets	Core schema: multi-tenant branches, staff roles, service catalogue, queue tickets
0002_staff_invite_link.sql	staff_invites	Tokenised invite URLs for staff onboarding without manual account creation
0003_staff_status_log.sql	staff_status_log	Clock-in/out tracking, shift history, break logging
0004_arrival_tracking.sql	arrival_events	Tracks when customers physically arrive vs. when they checked in remotely
0005_personal_tables.sql	tasks, focus_sessions, docs	Personal productivity mode: deep work timer, task lists, document storage
0006_public_bookings.sql	bookings, availability_slots	Public-facing booking page: time slot management, booking confirmation
0007_billing.sql	subscriptions, billing_events	Stripe subscription state, plan tier tracking, upgrade/downgrade events
0008_stations.sql	stations, station_events	Counter/station management, staff-to-station assignment
0009_sla.sql	sla_rules, sla_breaches	SLA threshold configuration, automatic breach detection and escalation
0010_station_events_insert_rls.sql	—	RLS policy fix: allows station event inserts from authenticated staff
0011_customers.sql	customers, customer_events, customer_profiles	Customer profiles, event timeline, AI persona storage
0012_messaging.sql	channel_connections, message_log	Multi-channel config (WA/FB/IG/email), outbound message history
0013_bounce_escalation.sql	bounce_log	Detects unanswered call notifications and auto-escalates tickets
0014_ticket_email.sql	—	Adds email column to tickets for email-based notifications
0015_satisfaction_scores.sql	satisfaction_scores	Post-visit 1–5 star ratings linked to customer, ticket, and staff
0016_notifications_log.sql	notifications_log	Full audit log of every outbound WhatsApp/SMS notification sent

All tables use **Row Level Security (RLS)** enforced at the PostgreSQL level. Policies verify that the authenticated user (auth.uid()) belongs to the branch they are accessing, preventing any cross-tenant data leakage even if the client is compromised.

5. Core Features — Implemented & Deployed

● Real-Time Queue Management

Walk-ins and bookings merge into one ordered queue via Supabase Realtime WebSocket subscriptions. Staff see live updates — new tickets, status changes, reassignments — without refreshing the page. Supports split-lane mode (Fast Lane / Complex Lane) and complexity scoring (Simple / Standard / Complex / Extended).

● QR Code Self-Check-In (Multilingual Kiosk)

Customers scan a branch QR code, land on CustomerCheckIn.jsx, select their language (English, Bahasa, Arabic, Chinese, Tamil, Urdu), choose a service, and enter their phone number. A ticket is issued automatically. The check-in flow is fully self-service — no staff intervention required.

● AI Customer Personas

When a customer joins the queue, findOrCreateCustomer() resolves their identity by phone number. generatePersona() calls OpenAI GPT-4o-mini with their visit history, satisfaction scores, and Freshdesk support tickets to produce a plain-language staff briefing displayed on the queue dashboard before the customer reaches the counter. New vs. returning detection is instant.

● WhatsApp / SMS Notifications via Twilio

The send-notification Deno Edge Function sends templated WhatsApp messages via Twilio for four events: check-in confirmation, 'you're next' call, post-service thank-you, and prayer-pause notice. A dry-run mode logs would-be messages to notifications_log without sending, enabling testing without Twilio credits.

● Document Checklists

When a booking is confirmed, checklists.js sends a WhatsApp message listing the documents the customer needs to bring. 50+ service templates are built in (tax returns, passport renewal, health screening, etc.). Each branch can customise the template list.

● Satisfaction Scores

When staff complete a ticket, a 5-emoji survey overlay appears (implemented in Queue.jsx as SatisfactionSurvey). One tap saves the score to satisfaction_scores with the ticket_id, customer_id, staff_id, and optional note. Scores accumulate on the customer profile and display as an all-time average.

● Islamic Mode — Prayer Time Auto-Pause

prayerTimes.js calculates all 5 daily prayer times for the branch's coordinates using sun-angle formulas. 10 minutes before each prayer, the queue auto-pauses, queued customers receive a WhatsApp prayer-pause notification, and the dashboard displays a prayer countdown. Jumu'ah (Friday prayer) is also handled.

● Autopilot Queue Pacing

autopilot.js monitors queue depth and average service time to calculate an adaptive calling interval. When staff are behind, the pace slows automatically. When the queue grows, it speeds up. The engine also routes each ticket to the least-loaded eligible staff member via autoAssign.js.

● SLA Monitoring & Escalation

sla.js checks all active tickets against configurable time thresholds. Breaches are written to sla_breaches and trigger escalation flags on the ticket. The Manager Intelligence panel surfaces which staff and service types generate the most SLA breaches.

● Stripe Subscription Billing

Four tiers (Essential RM29, Professional RM59, Executive RM99, Manager RM149). Checkout is handled via the create-checkout-session Edge Function. Subscription state is synced via stripe-webhook. Customers self-manage via the customer-portal-session Edge Function.

● Manager Intelligence Dashboard

managerIntel.js aggregates break patterns, anomaly detection (unusual wait spikes), and wellness scoring across all staff. wellness.js computes a per-staff wellness index from serving pace, break regularity, and error rates to surface burnout risk before it affects service quality.

6. Supabase Edge Functions (Server-Side Logic)

Edge Functions run on Deno at Supabase's edge network. They handle operations that must not run client-side: API secrets, Stripe webhook verification, Twilio credentials, and service-role database access.

Function	Runtime	External API	Purpose
send-notification	Deno/TypeScript	Twilio WhatsApp	Sends confirm/call/thanks/prayer_pause WhatsApp messages. Dry-run
send-email	Deno/TypeScript	SMTP (branch config)	Sends email notifications using branch-configured SMTP credentials s
ai-assist	Deno/TypeScript	OpenAI GPT-4o-mini	Generates customer AI personas from visit history and event timeline.
admin-stats	Deno/TypeScript	Supabase service role	Aggregates cross-branch statistics for admin dashboard with service-r
create-checkout-session	Deno/TypeScript	Stripe API	Creates Stripe Checkout session for subscription purchase, returns re
customer-portal-session	Deno/TypeScript	Stripe API	Creates Stripe Customer Portal session for self-service subscription m
stripe-webhook	Deno/TypeScript	Stripe API	Receives and verifies Stripe webhook events; updates subscriptions t

7. Key Code Samples

Every time a customer checks in, the system resolves their identity by phone number and creates a profile if one does not exist. This enables visit history accumulation and AI persona generation.

```
export async function findOrCreateCustomer(branchId, phone, name = null) { const normalised =
normalisePhone(phone); // Try to find existing customer const { data: existing } = await supabase
.from("customers") .select("*") .eq("branch_id", branchId) .eq("phone", normalised)
.maybeSingle(); if (existing) return existing; // Create new customer profile const { data:
created } = await supabase .from("customers") .insert({ branch_id: branchId, phone: normalised,
name }) .select("*") .single(); return created; }
```

The Deno Edge Function hydrates the ticket, renders the correct message template, and sends via Twilio. All results are logged to notifications_log regardless of delivery success.

```
const TEMPLATES = { confirm: ({ token, branchName, serviceName }) => `${branchName}\n\nYou're
checked in. Ticket ${token} for ${serviceName}. We'll WhatsApp again when you're up.`, call: ({
token, branchName }) => `${branchName}\n\nYou're up – please come to the counter. Ticket
${token}.`, thanks: ({ token, branchName }) => `${branchName}\n\nThanks for visiting. (Ticket
${token})`, }; // Send via Twilio const params = new URLSearchParams({ From: from, //
"whatsapp:+14155238886" To: `whatsapp:${normalisePhone(ticket.customer_phone)}`, Body: message,
}); const res = await fetch(twilioUrl, { method: "POST", headers: { "Authorization": "Basic " +
btoa(`${sid}:${auth}`) }, body: params, });
```

```
export async function saveScore({ branchId, ticketId, customerId, staffId, score, note }) { const
{ error } = await supabase .from("satisfaction_scores") .insert({ branch_id: branchId, ticket_id:
ticketId || null, customer_id: customerId || null, staff_id: staffId || null, score, // 1-5 note:
note?.trim() || null, }); if (error) throw error; } export function scoreEmoji(score) { return
["", "■", "■", "■", "■", "■", "■"][score] ?? "-"; }
```

Every table uses RLS to enforce branch isolation at the database level. Staff can only access their own branch's data.

```
-- Branch staff can read/write their own branch's satisfaction scores CREATE POLICY "branch staff
can manage scores" ON public.satisfaction_scores USING ( EXISTS ( SELECT 1 FROM public.staff s
WHERE s.branch_id = satisfaction_scores.branch_id AND s.user_id = auth.uid() ) ) WITH CHECK (
EXISTS ( SELECT 1 FROM public.staff s WHERE s.branch_id = satisfaction_scores.branch_id AND
s.user_id = auth.uid() ) );
```

8. Deployment Evidence

Evidence	Detail
Live URL	https://azqueue.io — accessible to anyone, fully functional
GitHub Repository	https://github.com/ahmedhamid2000422-eng/azqueue-saas-app
GitHub Branch	main — auto-deploys to Vercel on every push
Vercel Project	https://vercel.com/ahmedhamid2000422-engs-projects/azqueue-saas-app
Deployment Method	Git push to main → Vercel detects → Vite build → CDN deploy (~45s)
Backend	Supabase Cloud project — PostgreSQL + Auth + Realtime + Edge Functions
Database Migrations	16 SQL files in /supabase/migrations/, applied sequentially
Edge Functions	7 Deno functions deployed to Supabase Functions endpoint
Custom Domain	azqueue.io — registered and DNS-configured via Vercel
SSL Certificate	Auto-provisioned by Vercel via Let's Encrypt
Environment Variables	VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY, VITE_OPENAI_API_KEY set in Vercel dashboard
Commit History	Continuous development commits visible on GitHub repository

9. Project Scope — What Was Built

Component	Count	Evidence Location
React components / pages	30+	src/pages/, src/modes/, src/components/
JavaScript library modules	20	src/lib/
SQL migration files	16	supabase/migrations/
Deno Edge Functions	7	supabase/functions/
External API integrations	5	OpenAI, Twilio, Stripe, Freshdesk, Meta WhatsApp
Database tables	16+	Across all migrations
RLS policies	30+	Every table has branch-level isolation
Languages used	4	JavaScript/JSX, TypeScript, SQL, CSS (Tailwind)
Deployment environments	2	Vercel (frontend), Supabase Cloud (backend)
Design tokens	15+	tailwind.config.js
npm packages	15+	package.json

This document was produced as part of an APEL.C submission to demonstrate that AzQueue is a real, deployed, full-stack software system — not a concept, wireframe, or prototype. All source code is available for inspection at the GitHub repository listed on the cover page.

Ahmed Hamid · ahmedhamid2000422@gmail.com · May 2026